

方块游戏

Barchin 和 Charos 正在一个由 $2N \times 2M$ 正方形单元格组成的网格上玩游戏。从上到下，行的编号依次为 0 到 $2N - 1$ ；从左到右，列的编号依次为 0 到 $2M - 1$ 。对于 $0 \leq i < 2N$ 和 $0 \leq j < 2M$ ，我们将 i 行和第 j 列的单元格表示为 (i, j) 。

Barchin 依次给 Charos $N \cdot M$ 个**方块**。每个方块都是一个 2×2 的正方形，由四个 1×1 **小方块**组成。Barchin 已经将方块中的每个小方块涂成黑色或白色，并且保证**至少有一个小方块是白色的**。

在不知道之后会收到哪些方块的情况下，Charos 必须在**收到每个方块后立即**将其放置在网格上。方块不能旋转。每个方块必须完全放置在网格内，且正好覆盖四个网格单元格。此外，每个方块左上角的小方块必须覆盖一个行坐标和列坐标均为偶数的单元格。网格中的每个单元格最多只能被一个方块覆盖。

如果在某次放置一个方块后，存在一个 2×2 的正方形，其中四个单元格被四个黑色小方块覆盖，则 Barchin 获胜。形式上，如果单元格 (a, b) 、 $(a + 1, b)$ 、 $(a, b + 1)$ 、 $(a + 1, b + 1)$ 都被黑色小方块覆盖（其中 $0 \leq a < 2N - 1$ 且 $0 \leq b < 2M - 1$ ），则 Barchin 获胜。其中 a 和 b 不必是偶数。

如果 Charos 放置了所有 $N \cdot M$ 个方块，而 Barchin 始终未获胜，则 Charos 获胜。注意，放置 $N \cdot M$ 个方块将完全覆盖网格。

你的任务是为 Charos 制定赢得游戏的策略。可以证明，在给定约束下，无论之后收到的方块如何着色，Charos 总能通过正确放置方块以保证获胜。

实现细节

你要实现以下两个函数：

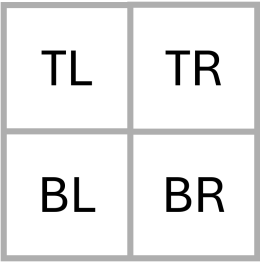
```
void init(int N, int M)
```

- N ：网格中行数的一半。
- M ：网格中列数的一半。
- 该函数在每个测试用例中只调用一次，即在你的程序执行开始时调用。

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL 、 TR 、 BL 、 BR ：分别为当前方块左上角、右上角、左下角和右下角小方块的颜色，如下图所示。每个值要么是 0 （白色），要么是 1 （黑色）。

- 在首次调用 `init` 之后，每个测试用例将调用此函数恰好 $N \cdot M$ 次。



此函数应返回一对整数 (i, j) ，其中 i 是该方块左上角小方块应放置的单元格的行坐标， j 是该单元格的列坐标。 i 和 j 都必须是偶数，并且该方块覆盖的 2×2 区域不得与任何先前放置的方块重叠。

如果 `receive_block` 返回的整数对不满足这些要求，或者放置方块后，一个 2×2 正方形单元格完全被黑色小方块覆盖，评测程序将立即终止你的程序，并且该测试用例的判定结果为 `Output isn't correct`。

评测程序的行为**不具备自适应性**。这意味着在调用 `init` 之前，Barchin 交给 Charos 的方块顺序就已经固定了。

约束条件

对于每个方块，令 S 为其包含的四个小方块中黑色小方块的数量。即 $S = TL + TR + BL + BR$ 。

- $1 \leq N, M \leq 100$
- 对于每个方块， $0 \leq S \leq 3$ 。

子任务

子任务	分数	额外的约束条件
1	6	每个方块的 $S = 1$ ，且 $N = 2$ 。
2	16	每个方块的 $S = 3$ 。 $N = M$ ， N 为偶数，且四种可能的方块着色方案每一种都恰好出现 $\frac{N^2}{4}$ 次。
3	10	每个方块的 $S = 1$ 。
4	29	每个方块的 $S \leq 2$ 。
5	39	没有额外的约束条件。

例子

考虑一次游戏，其中 $N = 1$ ， $M = 2$ ，因此网格有 2 行和 4 列。评测程序首先调用：

```
init(1, 2)
```

初始状态下，所有单元格均为空。网格如下所示：

	0	1	2	3
0				
1				

需要放置 $N \cdot M = 2$ 个方块。假设 Barchin 给出一个方块，其中包含三个黑色小方块和一个位于右上角的白色小方块。评测程序调用：

```
receive_block(1, 0, 1, 1)
```

Charos 决定将此方块放置在网格的左侧，返回(0,0)。

现在的网格看起来是这样的：

	0	1	2	3
0				
1				

Barchin 随后给出另一个方块，其左上角为白色，其余三个小方块为黑色：

```
receive_block(0, 1, 1, 1)
```

在剩余的网格单元格中，唯一行和列均为偶数且能作为 2×2 方块左上角的单元格是 (0,2)，因此 Charos 返回 (0,2)。最终的网格如下所示：

	0	1	2	3
0				
1				

因为没有 2×2 方格被黑色小方块完全覆盖，所以 Charos 成功放置了所有方块，Barchin 始终未获胜。Charos 赢得游戏。

评测程序示例

输入格式：

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

输出格式：

```
R[0] C[0]
R[1] C[1]
...
R[NM-1] C[NM-1]
```

这里， $R[k]$ 和 $C[k]$ 是第 k 次调用 `receive_block` 返回的一对整数。